

Isochronous Fixed-Weight Sampling in Hardware

Lightweight, High-Performance and Side-Channel Secure

Adrian Marotzke^a 

NXP Semiconductors, Hamburg, Germany

Abstract. We present hardware implementations of the recently proposed isochronous fixed-weight sampling algorithm by Décio Luiz Gazzoni Filho, Tomás S. R. Silva, Julio López (CiC vol. 1, 2024) and apply them to the post-quantum cryptographic algorithms NTRU-HPS, Streamlined NTRU Prime and Classic McEliece. We offer multiple implementations, optimized for different targets: A high-area high-performance implementation, a lightweight low-area implementation, as well as a side-channel secure implementation using gadget-based masking. We verify the security of our masked implementation using the PROLEAD leakage detection tool. We show that the sampling algorithm results in highly efficient and effective implementations for the NTRU-like schemes, outperforming existing hardware implementations for fixed-weight sampling.

Keywords: Post-quantum cryptography · Sampling · Hardware Implementation · NTRU · Streamlined NTRU Prime · Classic McEliece · FPGA · Masking · Gadget-based Masking

1 Introduction

Post-quantum cryptography (PQC) is now widespread, with multiple standards available, and numerous ciphers implemented in cryptographic libraries and protocols. NTRU-HPS, Streamlined NTRU Prime and Classic McEliece are three ciphers that, despite not being part of any official government standard, have seen deployment, e.g. in OpenSSH [bri25, Ope22] and VPN software [Mul23] as well as standardization in other forms [FMJ25]. All three algorithms share a common component, namely they require the sampling of a so-called fixed-weight vector or polynomial. Efficient implementation of this routine is thus a key part of accelerating these PQC algorithms. Furthermore, hardware implementations are also highly beneficial for widespread adoption, as dedicated hardware can dramatically increase performance, reduce power consumption as well as improve an implementation's resistance to side-channel attacks (SCA). Décio Luiz Gazzoni Filho, Tomás S. R. Silva and Julio López recently proposed a new algorithm for fixed-weight sampling, and their software implementation outperforms existing methods [FSL24]. The question thus arises whether their algorithm also shows similar improvements for hardware implementations.

Our Contribution

- A high-speed hardware implementation of the fixed-weight sampling algorithm from [FSL24], optimized for maximum performance.
- A low-area hardware implementation of [FSL24], optimized for minimal area consumption. This variant requires no memory or DSP.

E-mail: adrian.marotzke@nxp.com (Adrian Marotzke)

^aAdditional email: adrian.marotzke@tuhh.de



- An arbitrary-order masked hardware implementation of [FSL24] using HPC2 gadget-based masking. The side-channel security has been verified with the PROLEAD tool [MM22]. To our knowledge, this is the first side-channel protected hardware implementation of fixed-weight sampling for lattice schemes in the literature.
- We apply and benchmark our implementations to NTRU-HPS, Streamlined NTRU Prime and Classic McEliece on FPGAs.
- We release the HDL source code of our implementation under an open-source license. It is available at <https://github.com/NXP-Research/IsochronousSampling>.

2 Background

2.1 Fixed-Weight Sampling

Fixed-weight sampling is a core component of many PQC algorithms, in particular lattice- and code-based algorithms. Fixed-weight in this context means that a vector (or a polynomial) is randomly sampled with only a specific number of non-zero entries, typically -1 or 1 (in which case it is called ternary sampling). For example, during Streamlined NTRU Prime key generation and encapsulation, a polynomial of degree n with exactly w coefficients being 1 or -1 is sampled (all other coefficients are zero), with the value of n and w depending on the exact parameter set [BCLvV17, BBC⁺20]. Similarly, NTRU-HPS requires the sampling of a random polynomial of degree $n - 1$ with exactly $w/2$ coefficients equal to 1 , $d/2$ coefficients equal to -1 and all other coefficients equal to zero [CDH⁺20], again with the value of n and w dependent on the parameter set. Classic McEliece samples a binary fixed-weight vector, where the vector of size n has exactly w entries with the value of 1 and the rest 0 [ABC⁺22].

For both lattice algorithms mentioned above, the authors recommend to use a sorting algorithm as a permutation, in order to randomize a vector with the desired weight. One key reason for this is that a sorting algorithm can be implemented in constant time, which is crucial for cryptographic algorithms. Other ways of shuffling such a vector, such as Fisher-Yates, are difficult to implement in a constant-time fashion and thus generally not suitable for cryptographic purposes [FSL24]. Nevertheless, Fisher-Yates shuffle has been used in hardware implementations of NTRU [ABPS23]. Sorting-based sampling has been widely used for hardware implementations, with various different sorting algorithms being presented, such as mergesort in [DMG21] and radix sort in [PMT⁺23, Mar25].

In [FSL24], Filho et al. presented a new approach for fixed-weight sampling, namely a constant-time rejection-sampling based shuffle. It offers better asymptotic complexity over the sorting-based methods, resulting in an overall improved runtime performance when benchmarking their software implementation. It also reduced the consumption of fresh randomness. Their algorithm consists of two parts, described in Algorithm 1 and 2. For the proof of correctness as well as a detailed explanation of them, we refer to their paper [FSL24].

In Algorithm 2, the number of -1 's is set implicitly by c_0 and c_1 , with the number of -1 's being equal to $n - c_0 - c_1$. With NTRU-HPS, the number of -1 's in the sampled vector is equal to the number of 1 's, allowing c_0 to simply be set to the specified value depending on the parameter set. However, this is not the case for Streamlined NTRU Prime where no such restriction exists. [FSL24] provided a suggestion for this, namely to sample an array of $n - c_0$ random bits and then setting c_1 to the Hamming weight of this array. This corresponds to a binomial distribution and is shown in Algorithm 3.

The fixed-weight sampling algorithm of [FSL24] as well as the sorting-based methods are best suited for PQC algorithms where the to-be-sampled vector is comparatively dense, i.e. where the number of set elements is of similar magnitude as the vector length. This

Algorithm 1 $\text{RejSamplingMod}(n)$: Generation of an array of unbiased uniformly random integers modulo $n - 1, n - 2, \dots, 2$ [FSL24].

Input n
Output $\mathbf{si}$ (output array of $n - 1$ integers, such that $\mathbf{si}[i] \sim \mathcal{U}(0, n - 1 - i)$)

```

1: for  $i$  from 0 to  $n - 2$  do
2:    $t[i] \leftarrow 2^L \bmod (n - 1 - i)$  ▷ Precomputation step, done prior
3: end for
4: for  $i$  from 0 to  $n - 2$  do
5:   repeat
6:      $x \xleftarrow{\$} 0, 1, \dots, 2^L - 1$  ▷ Sample a uniform random integer of  $L$  bits
7:      $s \leftarrow n - 1 - i$ 
8:      $m \leftarrow x \cdot s$ 
9:      $l \leftarrow m \bmod 2^L$  ▷ Reduction can be done via a bitmask
10:    until  $l \geq t[i]$ 
11:     $\mathbf{si}[i] \leftarrow \lfloor m/2^L \rfloor$  ▷ Divide using a right-shift by  $L$  bits
12: end for
13: return  $\mathbf{si}$ 

```

is the case for NTRU-HPS and Streamlined NTRU Prime. For example for the *sntrup761* parameter set, the length is $p = 761$ with a weight $w = 286$. Classic McEliece has also seen the use of sorting-based sampling [ZZC⁺23, DMG21, WSN18], though the vectors are sparser than the NTRU-like schemes, with e.g. a degree of 8192 and weight of 128. Due to the sparse vector in Classic McEliece, rejection-based approaches have also shown to be highly efficient, such as in [CCD⁺22]. Applying Algorithm 1 and 2 to Classic McEliece is straightforward: Algorithm 1 can be left unmodified, and in Algorithm 2 all lines related to c_1 as well as Line 10 are removed.

On the other hand, code-based PQC algorithms such as HQC and BIKE have sampled vectors that are even sparser than Classic McEliece, with one parameter set of HQC for example having a degree of 17669 and a weight of 75. In these cases, other sampling methods are faster than the method from [FSL24] or the sorting-based approaches from [BBC⁺20, CDH⁺20]. Instead, methods such as the sampling of the index of non-zero elements while rejecting index collisions are more performant [FSL24]. This was investigated in [FAB⁺25], which confirmed that the algorithm from [FSL24] is not suitable for HQC.

2.2 Side-Channel Attacks

In reality, cryptographic algorithms must run on actual, physical devices. This opens up the possibility of *side-channel attacks* (SCA), where an attacker can measure e.g., the timing or power consumption of a device running a cryptographic algorithm, and attempt to extract secret data from said measurements. The authors of [FSL24] had already taken timing side-channels into account during their work, and their algorithm ensures that an attacker cannot extract any secret data based on timing information. However, they did not address more powerful side-channel attacks such as differential power analysis [KJJ99]. In [KAA21], the authors demonstrate successful single-trace side-channel attacks against the software implementations of fixed-weight sampling of both NTRU-HPS and Streamlined NTRU Prime, showing that this step is indeed vulnerable.

To properly model the security of side-channel protected designs, [ISW03] introduced the d -probing model. This allows an attacker to insert up to d probes in the circuit, reading the contents of the wires. However, this abstraction did not model certain physical phenomena of circuits, such as glitches or transitions. It was extended to the robust d -probing model in order to incorporate these aspects [BGI⁺18, FGM⁺18].

Algorithm 2 $\text{Shuffle}(n, c_0, c_1)$: Ternary fixed-weight sampling by shuffling [FSL24].

Input n
Input c_0, c_1 (the number of coefficients equal to 0 and 1, respectively)
Output $\mathbf{v}$ (output array of $n - 1$ integers)

```

1:  $\mathbf{si} \leftarrow \text{RejSamplingMod}(n)$ 
2: for  $i$  from 0 to  $n - 2$  do
3:   if  $\mathbf{si}[i] < c_0$  then                                 $\triangleright$  These if-clauses must be done in constant time
4:      $\mathbf{v}[i] \leftarrow 0$ 
5:      $c_0 \leftarrow c_0 - 1$ 
6:   else if  $\mathbf{si}[i] < c_0 + c_1$  then
7:      $\mathbf{v}[i] \leftarrow 1$ 
8:      $c_1 \leftarrow c_1 - 1$ 
9:   else
10:     $\mathbf{v}[i] \leftarrow -1$ 
11:   end if
12: end for
13: return  $\mathbf{v}$ 

```

Algorithm 3 $\text{Sample}_{c_1}(n, c_0)$: Sample c_1 according to a binomial distribution [FSL24].

Input n
Input c_0
Output c_1

```

1:  $c_1 \leftarrow 0$ 
2: for  $i$  from 0 to  $n - c_1$  do
3:    $x \xleftarrow{\$} 0, 1$                                  $\triangleright$  Sample a uniform random bit
4:    $c_1 \leftarrow c_1 + x$ 
5: end for
6: return  $c_1$ 

```

As a countermeasure to advanced SCAs, *masking* has proved to be an efficient countermeasure. Masking, based on Shamir’s secret sharing, splits sensitive data into so-called shares, and operates on these individual uniform random shares. Boolean masking, for example, splits the secret value x into $d + 1$ shares $x^{(i)}$, such that $x = \bigoplus_{i=0}^d x^{(i)}$. We use d as the masking order. In the robust d -probing model, the attacker can insert d probes, but since each sensitive value is split up into $d + 1$ shares, the attacker does not gain any information on the secret values. A higher masking order thus corresponds to a circuit more secure against SCAs. We will write $x^{(0:d)}$ to indicate a masked variable.

Side-channel secure fixed-weight sampling using masking was extensively studied in [KLRBG23], though as it predates [FSL24], they could not take the new algorithm into account. While [KLRBG23] focused mainly on software implementations, they also studied masked hardware implementations of fixed-weight sampling for the code-based scheme BIKE [ABB⁺22]. However, as the sampled vector for BIKE is highly sparse, their method is not suitable for the dense fixed-weight sampling used in schemes like NTRU-HPS and Streamlined NTRU Prime.

2.3 Gadget-Based Masking

The approach of gadget-based masking is to replace individual hardware gates with secure masked version, called gadgets. These gadgets are carefully designed to not leak their inputs and outputs as well as any intermediates via side-channels. An example of a gadget is Hardware Private Circuit (HPC), proposed by [CGLS21]. The authors implement a

secure, masked AND gadget, which can be universally composed to build arbitrary circuits at any masking order. They propose two variants HPC1 and HPC2, with HPC2 requiring less fresh randomness per cycle. We will be using HPC2 throughout this paper. One of the main benefits of masking using HPC2 is its simplicity and composability: Unlike masking methods such as Threshold Implementations [NRR06], we can arbitrarily connect HPC2 gadgets together, and the resulting circuit will be secure. Furthermore, the masking order can be easily adjusted via a single parameter, and does not require any design changes. Gadget-based designs can also be formally verified using tools such as VERICA [RBFSG22], though this may be infeasible if the design is too large.

Gadget-based masking has already been used to secure full PQC hardware implementations, such as Streamlined NTRU Prime [LMRBG24] and Kyber [BLH⁺25]. However, it has not yet been applied to the task of fixed-weight sampling. The authors in [LMRBG24] only implemented the decapsulation of Streamlined NTRU Prime, which does not require fixed-weight sampling.

2.4 PROLEAD

PROLEAD is an automated tool designed to analyze the side-channel security of masked hardware circuits [MM22]. Rather than formal verification, it uses logic simulations to evaluate the security, allowing PROLEAD to handle larger and more complicated circuits. PROLEAD only requires the gate-level netlist of a design, and can also properly deal with the presence of glitches and transition leakages in the robust d -probing model. While simulating the gate-level netlist, PROLEAD performs a G -test [McD14] to check for statistical independence between fixed and random inputs. The value of 10^{-5} is used as threshold to indicate the presence leakages, in a similar fashion to the widely used t-test [SM15]. During the simulation, the tool also outputs the statistical confidence of the results, allowing the user to gauge whether more simulations are required. The tool is available at <https://github.com/ChairImpSec/PROLEAD>. It has proven itself to be able to successfully detect vulnerabilities in masked circuits [MM22, RM25].

3 Hardware Implementation

In this section, we will present our high-performance and low-area hardware implementation for fixed-weight sampling, and discuss implementation-specific aspects.

3.1 High-Performance Implementation

Our high-speed design is optimized for highest performance. For this, we fully pipeline the second for-loop of Algorithm 1. We choose $L = 16$ for NTRU-based designs, the same as [FSL24], as they showed that it strikes a good balance between the chance of rejection and randomness consumption. For the Classic McEliece design, we choose $L = 19$. In addition, multipliers for these values of L are readily available in FPGAs in the form of DSP units [Xil18].

To prevent the pipeline from stalling, we check if a sampled value is valid or must be rejected (Line 10 in Algorithm 1). If valid, we store the sampled value in RAM at the corresponding address i (Line 11). However, if the value is rejected, we instead store the index i in an extra FIFO memory. Once the for-loop is complete, we read the indices from the FIFO, and repeat the sampling for these values. In case of another rejection, the index is stored once again in the FIFO. This repeats until the FIFO is completely empty and the pipeline has been flushed, at which point $\mathbf{si}$ is valid and Algorithm 1 is complete.

One question here is how deep the FIFO should be. Should the number of rejections exceed the FIFO depth, then the entire algorithm must restart. We compute an upper

bound of probability of this occurring in the following equation:

$$P(\text{rej} > t) = \sum_{k=t+1}^{n-1} \binom{n-1}{k} \cdot (1 - p_{\min})^k \cdot (p_{\min})^{n-1-k} \quad (1)$$

Where $\binom{n-1}{k}$ is the binomial coefficient, t is the FIFO depth and $p_{\min}$ is the minimum probability of a non-rejection sample for *any* index for a given L and vector length $n - 1$

Based on Equation 1, we write a Sagemath script that calculates the probability for different FIFO depths and vector lengths of NTRU-HPS and Streamlined NTRU Prime (see Appendix A). We choose a FIFO depth of 128. This gives a worst-case probability of the FIFO filling up of $2^{-39.27}$, which is the case for the Streamlined NTRU Prime parameter set *sntrup1277*. All other NTRU and NTRU Prime parameter sets have a lower probability. In practice of course, the actual probability of the FIFO being full is even lower, as the above use the worst-case rejection probability. What's more, in the unlikely event that the number of rejected samples exceeds 128 and the FIFO is full, then the entire algorithm simply aborts and starts from the beginning.

Our variant of *RejSamplingMod*, called *PipeRejSamplingMod* is depicted in Algorithm 4. Once **si** has been generated, the values can be read from memory sequentially and compared with the values c_0 and $c_0 + c_1$, as per Algorithm 2. This is straightforward in hardware, and can be done with adders and multiplexers in a single clock cycle to ensure constant-time execution. Furthermore, as we only perform a single pass over **si**, we can use a RAM with separated read and write ports to already start the next iteration of Algorithm 4. This allows us to interleave the operation of Algorithm 4 and 2, saving further time. In the case of Streamlined NTRU Prime, we first also perform Algorithm 3 to get the value of c_1 , before starting Algorithm 2.

3.2 Low-Area Implementation

For our low-area implementation, minimizing the total area is our priority. In particular, the memory to store **si** and the multiplication of $x \cdot s$ in Algorithm 1 are our main focus. There are two important details here: First is that in the $x \cdot s$ multiplication (Line 8, Algorithm 1), only the x is secret, with s being a public value. And secondly, the fact that Algorithm 2 reads **si** sequentially in a single pass.

The fact that s is public allows us to replace the $x \cdot s$ multiplication with an iterative shift-and-add multiplication, using a simple adder. This does not expose a timing side-channel, as the timing variability depends solely on number of set bits of s , which is public. To be precise, the shift-and-add multiplication takes a number of cycles equal to the hamming weight of s . In case of a rejection, we simply repeat the process for the same index immediately, with a new random x . We do not have any pipelining here, so this is straightforward.

Our next optimization is the memory to store **si**. There is no requirement in Algorithm 2 for **si** to be computed in its entirety before beginning Algorithm 2. As **si** is read sequentially in a single pass, we remove the memory to store **si** entirely, and instead compute each value one-at-a-time when needed. We can then immediately compute the corresponding output **v**. The single value of **si** can then be discarded, as it is no longer needed. This variant, which we call *IterativeShuffle*, is depicted in Algorithm 5. As we output each value of **v** instead of buffering them, no memory cells (e.g. BRAM, SRAM, LUTRAM) are needed in the entire implementation.

Algorithm 4 PipeRejSamplingMod(n): Generation of an array of unbiased uniformly random integers modulo $n - 1, n - 2, \dots, 2$, in a manner suitable for pipelining in hardware.

Input n
Output $\mathbf{si}$ (output array of $n - 1$ integers, such that $\mathbf{si}[i] \sim \mathcal{U}(0, n - 1 - i)$)

```

1: for  $i$  from 0 to  $n - 2$  do
2:    $t[i] \leftarrow 2^L \bmod (n - 1 - i)$  ▷ Precomputation step, done prior
3: end for
4: for  $i$  from 0 to  $n - 2$  do ▷ All operations in this loop can be fully pipelined
5:    $x \xleftarrow{\$} 0, 1, \dots, 2^L - 1$ 
6:    $s \leftarrow n - 1 - i$ 
7:    $m \leftarrow x \cdot s$ 
8:    $l \leftarrow m \bmod 2^L$ 
9:   if  $l \geq t[i]$  then
10:     $\mathbf{si}[i] \leftarrow \lfloor m/2^L \rfloor$ 
11:   else
12:     $\mathbf{f}$  enqueue  $i$  ▷ Store index  $i$  in the FIFO  $\mathbf{f}$ 
13:   end if
14: end for
15: wait until pipeline is flushed
16: if  $\mathbf{f}$  is empty then
17:   return  $\mathbf{si}$ 
18: else
19:   repeat ▷ All operations in this loop can be fully pipelined
20:     $x \xleftarrow{\$} 0, 1, \dots, 2^L - 1$ 
21:     $i \leftarrow \mathbf{f}$  dequeue ▷ read index  $i$  from the FIFO  $\mathbf{f}$ 
22:     $s \leftarrow n - 1 - i$ 
23:     $m \leftarrow x \cdot s$ 
24:     $l \leftarrow m \bmod 2^L$ 
25:    if  $l \geq t[i]$  then
26:       $\mathbf{si}[i] \leftarrow \lfloor m/2^L \rfloor$ 
27:    else
28:       $\mathbf{f}$  enqueue  $i$ 
29:    end if
30:    wait until pipeline is flushed
31:  until  $\mathbf{f}$  is empty
32:  return  $\mathbf{si}$ 
33: end if

```

4 Masked Hardware Implementation

In this section, we will present our masked implementation for fixed-weight sampling. Its design is based on the low-area design (Algorithm 5) from Section 3.2, which we secure using gadget-based masking. The key feature is that the low-area design does not require a full multiplier, but can instead be implemented with an adder. Gadget-based masked adders have previously been presented in e.g. [BG22, LMRBG24, BLH⁺25]. Specifically, we re-use the 13-bit Sklansky adder from [LMRBG24] (which in turn is based on the Sklansky adder from [BG22]), which we modify to a width of 16-bit. [BG22] showed that the parallel-prefix adder design of Sklansky [Skl60] offer a good balance of area, latency and randomness consumption for masked designs. This masked variant, which we call MaskedIterativeShuffle, is shown in Algorithm 6.

Algorithm 5 IterativeShuffle(n, c_0, c_1): Ternary fixed-weight sampling by shuffling.

Input n
Input c_0, c_1 (the number of coefficients equal to 0 and 1, respectively)
Output $\mathbf{v}$ (output array of $n - 1$ integers)

```

1: for  $i$  from 0 to  $n - 2$  do
2:    $t[i] \leftarrow 2^L \bmod (n - 1 - i)$  ▷ Precomputation step, done prior
3: end for
4: for  $i$  from 0 to  $n - 2$  do
5:    $s \leftarrow n - 1 - i$ 
6:   repeat
7:      $x \xleftarrow{\$} 0, 1, \dots, 2^L - 1$ 
8:      $m \leftarrow x \cdot s$  ▷ Multiplication using an iterative shift-and-add
9:      $l \leftarrow m \bmod 2^L$ 
10:    until  $l \geq t[i]$ 
11:     $si \leftarrow \lfloor m/2^L \rfloor$  ▷ Compute single value si
12:    if  $si < c_0$  then
13:       $\mathbf{v}[i] \leftarrow 0$ 
14:       $c_0 \leftarrow c_0 - 1$ 
15:    else if  $si < c_0 + c_1$  then
16:       $\mathbf{v}[i] \leftarrow 1$ 
17:       $c_1 \leftarrow c_1 - 1$ 
18:    else
19:       $\mathbf{v}[i] \leftarrow -1$ 
20:    end if
21: end for
22: return  $\mathbf{v}$ 

```

A notable difference from the low-area design is that we choose $L = 15$ for the masked design. This means our masked iterative adder also only has an input width of 15-bit. Doing so allows us to use the 16th bit of the adder as the carry out, which in turn means we do not require an additional stage for the adder. This would increase the latency of the adder by two clock cycles (the HPC2 gadget has a latency of two clock cycles for a masked AND gate).

We implement the if-else-clause in Line 18 of Algorithm 6 using the same masked adder as above, together with a masked multiplexer. In addition, we store $c_0^{(0:d)}$ and $c_1^{(0:d)}$ as the two's complement of c_0 and c_1 respectively. This allows us to forgo masked subtraction and instead implement all arithmetic operations with the masked adder.

To update $c_0^{(0:d)}$ and $c_1^{(0:d)}$, we securely add 1 to both, and then feed the original values and the incremented ones into a masked multiplexer. This multiplexer uses the most significant bit of $si^{(0:d)} + c_0^{(0:d)}$ and $si^{(0:d)} + c_0^{(0:d)} + c_1^{(0:d)}$ respectively as the select signal. The result is then written back into $c_0^{(0:d)}$ and $c_1^{(0:d)}$. We use the same multiplexing to write $-1, 0$ or 1 into $\mathbf{v}^{(0:d)}[i]$. For the masked multiplexer itself, we reuse the design of [LMRBG24], which is open source and also uses the HPC2 gadget.

During the rejection sampling routine, we need to check whether $l^{(0:d)} \geq t[i]$ (Line 12, Algorithm 6). While the values of $t[i]$ are precomputed and public, $l^{(0:d)}$ is a secret and masked value. We implement this check as follows: using the masked adder, we subtract the value of $t[i]$ from $l^{(0:d)}$ by adding the two's complement of $t[i]$. We then *unmask* the most significant bit, which is the 16th bit and is the carry-out of the adder. This bit is then used to decide whether to reject the sampled integer or to accept it, as it is zero if $l^{(0:d)} \geq t[i]$ and one otherwise. Despite the unmasking, this does not reveal any information on the value of $l^{(0:d)}$ (and thus $m^{(0:d)}$), other than whether it is smaller or larger than $t[i]$.

Algorithm 6 MaskedIterativeShuffle(n, c_0, c_1): Ternary fixed-weight sampling by shuffling, using gadget-based masking.

Input n
Input d (the masking order)
Input c_0, c_1 (the number of coefficients equal to 0 and 1, respectively)
Output $\mathbf{v}^{(0:d)}$ (masked output array of $n - 1$ integers)

```

1: for  $i$  from 0 to  $n - 2$  do
2:    $t[i] \leftarrow 2^L \bmod(n - 1 - i)$  ▷ Precomputation step, done prior
3: end for
4:  $c_0^{(0:d)} \leftarrow -c_0$  ▷ Stored as negative number with shares  $c_0^{(1:d)}$  being zero
5:  $c_1^{(0:d)} \leftarrow -c_1$ 
6: for  $i$  from 0 to  $n - 2$  do
7:    $s \leftarrow n - 1 - i$ 
8:   repeat
9:      $x^{(0:d)} \xleftarrow{\$} 0, 1, \dots, 2^L - 1^{(0:d)}$  ▷ Sample a random masked integer
10:     $m^{(0:d)} \leftarrow x^{(0:d)} \cdot s$  ▷ Multiplication using a masked shift-and-add
11:     $l^{(0:d)} \leftarrow m^{(0:d)} \bmod 2^L$ 
12:    until  $l^{(0:d)} \geq t[i]$  ▷ See text on details for this step
13:     $\text{si}^{(0:d)} \leftarrow \lfloor m^{(0:d)} / 2^L \rfloor$ 
14:     $c_0'^{(0:d)} \leftarrow c_0^{(0:d)} + 1$ 
15:     $c_1'^{(0:d)} \leftarrow c_1^{(0:d)} + 1$ 
16:     $\text{sic0}^{(0:d)} \leftarrow \text{si}^{(0:d)} + c_0^{(0:d)}$ 
17:     $\text{sic0c1}^{(0:d)} \leftarrow \text{sic0}^{(0:d)} + c_1^{(0:d)}$ 
18:    if  $\text{sic0}^{(0:d)} < 0$  then ▷ Implemented using masked multiplexer
19:       $\mathbf{v}^{(0:d)}[i] \leftarrow 0$ 
20:       $c_0^{(0:d)} \leftarrow c_0'^{(0:d)}$ 
21:       $c_1^{(0:d)} \leftarrow c_1'^{(0:d)}$ 
22:    else if  $\text{sic0c1}^{(0:d)} < 0$  then ▷ Implemented using masked multiplexer
23:       $\mathbf{v}^{(0:d)}[i] \leftarrow 1$ 
24:       $c_0^{(0:d)} \leftarrow c_0^{(0:d)}$ 
25:       $c_1^{(0:d)} \leftarrow c_1'^{(0:d)}$ 
26:    else
27:       $\mathbf{v}^{(0:d)}[i] \leftarrow -1$ 
28:       $c_0^{(0:d)} \leftarrow c_0^{(0:d)}$ 
29:       $c_1^{(0:d)} \leftarrow c_1^{(0:d)}$ 
30:    end if
31: end for
32: return  $\mathbf{v}^{(0:d)}$ 

```

However, this can be easily deduced by observing the timing behavior of Algorithm 1 and is not a vulnerable side-channel, as was already analyzed in [FSL24]. As rejected values are completely discarded, an attacker does not learn any information on valid samples.

Algorithm 6 describes our masked implementation for NTRU-HPS, where the number of -1 is equal to the number of 1 in the sampled vector. For Streamlined NTRU Prime, where this is not the case, we take a slightly different approach. There, we only keep track of $c_0^{(0:d)}$. For each output value, we sample a new masked random bit $y^{(0:d)}$, which determines whether the output is -1 or 1 . We then securely multiplex between $(00)^{(0:d)}$ and $(y1)^{(0:d)}$ to get the output $v[i]^{(0:d)}$, using the most significant bit of $\text{si}^{(0:d)} - c_0^{(0:d)}$ as the selector. Although this requires us to sample a total of $n - 1$ random masked bits for $y^{(0:d)}$, we

do not need to implement a masked version of Algorithm 3, as well as any processing related to $c_1^{(0:d)}$. This includes the multiplexing for the second if clause (Line 22), and the computation of $si^{(0:d)} - c_0^{(0:d)} - c_1^{(0:d)}$. In all, this both simplifies and accelerates the algorithm, as well as requiring less randomness for the masked gadgets.

For Classic McEliece, Algorithm 6 can be simplified even more, as we only need to perform binary sampling. This allows us to remove all parts related to keeping track of $c_1^{(0:d)}$, as well as the second if-else clause in Line 22. Instead, we simply always multiplex between a $(0)^{(0:d)}$ and $(1)^{(0:d)}$, and output this value.

5 Evaluation

In this section, we implement and test our designs on Xilinx FPGAs, as well as compare them to the existing hardware implementations for fixed-weight sampling in the literature. We also analyze and verify the side-channel security of our masked design using the PROLEAD tool.

5.1 FPGA Evaluation

We implement our high-speed and low-area implementation for the NTRU-HPS parameter sets NTRU-HPS509, NTRU-HPS677 and NTRU-HPS821, for the Streamlined NTRU Prime parameter sets *sntrup761* and *sntrup1277*, as well as the Classic McEliece parameter set mceliece8192128. *sntrup761* is the recommended parameter set to use by the Streamlined NTRU Prime authors [BBC⁺20] and is also the set used by e.g. OpenSSH, while *sntrup1277* is the highest security parameter set. Our cycle benchmark numbers are the average of 1000 runs. We report area numbers for the Xilinx ZynqTM Ultrascale+TM FPGA, and we use VivadoTM v2023.1 for synthesis and implementation. Our area numbers do not include the cost of an RNG. The area and benchmark numbers are in Table 1, together with implementations in the literature on the same FPGA platform. [DMG21] and [ABPS23] do not report area numbers for their NTRU-HPS version of fixed-weight sampling, so we only list the cycle count and frequency.

While our low-area designs consistently use more LUT than our high-speed design for Streamlined NTRU Prime, it uses equal or slightly less LUT for Classic McEliece and NTRU-HPS. The low-area designs always consume fewer Flip-Flips (FF) and no BRAM or DSP for all parameter sets. The required clock cycles are significantly higher, up to $7.7\times$, though the low-area designs do achieve a slightly higher maximum clock frequency, except for Classic McEliece. For all our low-area designs, none of the LUT are used for distributed memory.

In comparison to the fixed-weight sampling designs in the literature for NTRU-HPS, our low-area design has a higher cycle count than the design from [DMG21], being about 1.9 times slower. Our design has a higher clock frequency, leading to the total execution time being lower. Compared to [ABPS23], we have both a lower cycle count and a higher clock frequency. However, this is not an apples-to-apples comparison. The sampling modules from [DMG21] and [ABPS23] are part a unified NTRU-HPS hardware implementation. If they were standalone, it is likely that their designs could also achieve higher clock frequencies. Our high-speed designs however, are by far the fastest across all parameter sets, having a cycle count that is between 3.9 to 8.2 times lower.

Compared to the Streamlined NTRU Prime fixed-weight sampling modules in the literature, our high-speed design outperforms all other designs by over one order of magnitude, while consuming less BRAM and, in all cases but one, less LUT and FF. Our high-speed design does however require one DSP, which no other design does. Our low-area design for Streamlined NTRU Prime is on par speed-wise with existing designs, while requiring no BRAM and fewer LUT and FF. The only exception is [Mar20], which

consumes even less LUT and FF, but requires one BRAM and is over 12 times slower than our low-area design.

Table 1: Our unmasked designs implemented on a Xilinx ZynqTM Ultrascale+TM FPGA, as well as a comparison with designs in the literature. The design from [CCD⁺22] is implemented on a Xilinx ArtixTM-7 FPGA, all others are on a Xilinx ZynqTM Ultrascale+TM. [CCD⁺22] has a 37% success probability and requires multiple passes in the case of failure. Our high-speed implementation is marked with TW-HS, our low-area is marked with TW-LA. n is the vector length and w is the target weight.

Cipher	Parameter	Design	LUT	FF	BR AM	DSP	Freq. [MHz]	Cycles	Time [μ s]
NTRU-HPS	$n=509, w=254$	TW-HS	248	128	0.5	1	425	518	1.219
		TW-LA	248	122	0	0	465	3826	8.228
		[ABPS23]	n/a	n/a	n/a	n/a	400	~4250	~10.6
	$n=677, w=254$	TW-HS	298	225	0.5	1	430	686	1.595
		TW-LA	296	128	0	0	470	5098	10.84
		[DMG21]	n/a	n/a	n/a	n/a	250	2678	10.71
	$n=821, w=510$	TW-HS	324	237	0.5	1	435	831	1.910
		TW-LA	314	129	0	0	475	6327	13.32
		[DMG21]	n/a	n/a	n/a	n/a	250	3343	13.37
SNTRUP	$n=761, w=286$	TW-HS	341	191	0.5	1	415	790	1.904
		TW-LA	438	138	0	0	440	5860	13.32
		[PMT ⁺ 23]	823	220	2	0	285	4837	16.97
		[DMG21]	1441	940	3.5	0	250	2762	11.05
		[Mar20]	231	87	1	0	279	49 400	177.1
		[Mar25]	844	285	2	0	400	4842	12.11
	$n=1277, w=492$	TW-HS	511	299	1	1	420	1323	3.15
		TW-LA	638	149	0	0	440	10 302	23.41
McEliece	$n=8192, w=128$	TW-HS	1933	380	7	1	380	8234	21.67
		TW-LA	1865	240	0	0	333	78 147	234.7
		[DMG21]	2533	1589	33	0	250	26 646	106.4
		[ZZC ⁺ 23]	2510	3887	20	0	300	24 598	81.99
		[WSN18]	583	411	20	0	250	147 505	590.0
		[CCD ⁺ 22]	304	148	2	0	262	1900	7.1

In addition, the two designs for *sntrup761* that are slightly faster than our low-area design, [Mar25] and [DMG21], require far more resources: [Mar25] requires nearly twice the LUT and FF, while [DMG21] consumes more than three times the LUT and six times the FF. Both also require BRAM, while our low-area design does not. In other words, duplicating our low-area design and allowing them both to operate in parallel would also result in a higher throughput while still consuming less FPGA resources than [Mar25] and [DMG21].

Our high-speed design for Classic McEliece has a lower cycle count than all other sorting-based designs in the literature [DMG21, ZZC⁺23, WSN18], as well as a higher clock frequency. However, it is slower than the rejection based approach from [CCD⁺22]. The comparison with [CCD⁺22] is made somewhat more complicated in two aspects: First, [CCD⁺22] only has a success probability of 37%. In other words, it is highly likely that multiple passes are required, which increases the average cycle count to obtain an output. The second aspect is that implementations for the ArtixTM-7 used by [CCD⁺22] generally have a lower maximum clock frequency than for our ZynqTM Ultrascale+TM. The design of [CCD⁺22] would have a higher clock frequency when implemented on a similar FPGA.

With regards to memory, our high-speed design for Classic McEliece requires less BRAM than any of the sorting-based designs, though it does require one DSP. In addition, our low-area design is the only implementation that does not require any BRAM at all. However, this comes at the cost of being 2.2, 2.9 and 33.7 times slower respectively than the designs from [ZZC⁺23], [DMG21] and [CCD⁺22], though still 2.5 times faster than the design from [WSN18].

5.2 FPGA Evaluation of Masked Design

We implement our masked implementation for the NTRU parameter sets NTRU-HPS509, NTRU-HPS677 and NTRU-HPS821, for the Streamlined NTRU Prime parameter sets *sntrup761* and *sntrup1277* as well as the Classic McEliece parameter set *mciece8192128*. We give numbers for masking order $d = \{1, 2, 3\}$. Our cycle benchmark numbers are the average of 300 runs. We report FPGA area numbers on the ZynqTM Ultrascale+TM FPGA, and we use VivadoTM v2023.1 for synthesis and implementation. The area and benchmark numbers are in Table 2. We separately list the randomness required to refresh the HPC2 gadgets each cycle, as the quality of this randomness is not as strict as that for a cryptographically secure RNG [CMM⁺24]. To our knowledge, this is the first masked fixed-weight sampling hardware implementation for these schemes in the literature, so we cannot compare it with other implementations, as [KLRBG23] only presented hardware implementations for BIKE. Instead, we compare it to our unmasked low-area design (where masking order $d = 0$).

Table 2: Our masked design for fixed-weight sampling, implemented on a Xilinx ZynqTM Ultrascale+TM FPGA for different masking orders d . For $d = 0$, we use the numbers of our low-area design. The column Rand. bits/cycle lists the maximum randomness per cycle needed to refresh the masking gadgets.

Cipher	Parameter	Order	LUT	FF	BR AM	DSP	Freq. [MHz]	Cycles	Time [μs]	Rand. bits/cycle
NTRU-HPS	$n = 509$	$d = 0$	248	122	0	0	465	3826	8.228	0
		$d = 1$	1596	1556	0	0	540	35 297	65.36	52
		$d = 2$	2847	3001	0	0	480	35 297	73.54	156
		$d = 3$	4796	5096	0	0	440	35 297	80.22	312
	$n = 677$	$d = 0$	296	128	0	0	470	5098	10.84	0
		$d = 1$	1727	1593	0	0	500	47 063	94.13	53
		$d = 2$	3023	3074	0	0	495	47 063	95.08	159
		$d = 3$	4993	5043	0	0	480	47 063	98.05	318
	$n = 821$	$d = 0$	314	129	0	0	475	6327	13.32	0
		$d = 1$	1748	1594	0	0	495	58 853	118.9	53
		$d = 2$	3049	3080	0	0	495	58 853	118.9	159
		$d = 3$	5013	5044	0	0	480	58 853	122.6	318
SNTRUP	$n = 761$	$d = 0$	438	138	0	0	440	5860	13.32	0
		$d = 1$	1609	1513	0	0	495	54 188	109.5	53
		$d = 2$	2841	2955	0	0	480	54 188	112.9	159
		$d = 3$	4752	4873	0	0	485	54 188	111.7	318
	$n = 1277$	$d = 0$	638	149	0	0	440	10 302	23.41	0
		$d = 1$	1738	1565	0	0	505	96 760	191.6	54
		$d = 2$	3017	3041	0	0	490	96 760	197.5	162
		$d = 3$	4971	5204	0	0	440	96 760	219.9	324
McEliece	$n = 8192$	$d = 0$	1865	240	0	0	333	78 147	234.7	0
		$d = 1$	2576	1734	0	0	365	812 423	2226	57
		$d = 2$	3942	3306	0	0	385	812 423	2110	171
		$d = 3$	6019	5410	0	0	405	812 423	2006	342

As expected, we can see in Table 2 that adding masking increases both the area (LUT and FF) consumption and cycle count compared to the unmasked design, for all three PQC algorithms. The area increases further when the masking order increases, though the cycle count stays the same. This behavior is due to the masking gadgets and was also observed in the gadget-based masked implementation of Streamlined NTRU Prime [LMRBG24]. Instead, the only factor influencing the cycle count is the parameter set, specifically the polynomial (or vector) degree.

Like the low-area designs, our masked implementations do not require any BRAM or DSP. For $d = 1$, the maximum clock frequency is higher than our low-area design for all parameter sets. However, as the masking order increases, the maximum clock frequency drops for NTRU-HPS and Streamlined NTRU Prime. This is due to the routing complexity of the masked Sklansky adder. Interestingly, this is not the case for Classic McEliece where the max clock frequency increases for higher masking order. The precise reason for this

behavior is not known, and we expect the clock frequency to decrease again for even higher masking orders.

Finally, the maximum randomness that is consumed per cycle for refreshing the HPC2 gadgets is almost entirely independent of the parameter set. It is slightly less for the NTRU-HPS509 parameter set, and only slightly more for the *sntrup1277* parameter set. This due to their polynomial degree being 9-bit and 11-bit respectively, rather than 10-bit. Classic McEliece also requires additional randomness, as its vector size is 14-bit.

5.3 Side-Channel Evaluation

Evaluating the side-channel security of a masking design is not trivial. However, using HPC2 gadgets has a benefit here, as they are trivially composable: Any combination of them is still secure. However, there are some things that could still damage security, for example by not properly connecting the randomness needed to refresh the gadget. For this reason, we use the side-channel analysis tool PROLEAD [MM22] to verify the first-order probing security of our masked iterative multiplier. This component is used for the fixed-weight sampling for all masked designs, and is one of two entities that operates on masked values. The other entity is the masked multiplexer used to implement the if-else clauses in Algorithm 6. We do not additionally analyze the masked multiplexer, as its side-channel security was formally verified in [LMRBG24].

As a sanity check that our setup is working correctly, we analyzed the masked Sklansky adder while disabling the randomness for refreshing the gadgets. The results of the PROLEAD analysis are shown in Figure 1. We can see that the $-\log_{10}(p)$ -value grows very quickly and reaches infinity after around 3800 simulations.

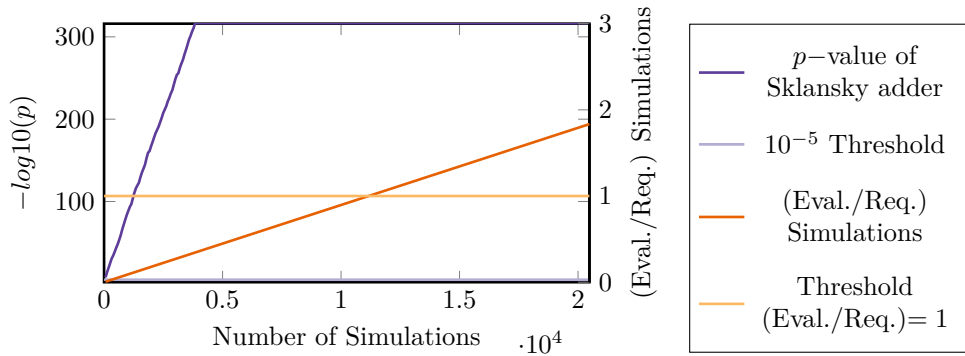


Figure 1: First-order Prolead simulation of the first-order 16-bit masked Sklansky adder, with the randomness for refreshing the masking gadgets disabled. As expected, the $-\log_{10}(p)$ -value rises very quickly, with the threshold crossed immediately. The $-\log_{10}(p)$ value reaches infinity after around 3800 simulations.

Our initial approach is to analyze the entire masked iterative multiplier as a single entity. However, this turns out to be infeasible for PROLEAD when using the default configuration: Even on a machine with 128GB of RAM, the tool runs out of memory after processing roughly 180 000 simulations. The required number of simulations for statistical confidence is also not reached. This incomplete run is depicted in Figure 2.

We repeat the simulation with PROLEAD running in *compact* mode. In compact mode, PROLEAD uses the Hamming weight of signals to check for leaks, rather than the full signal value. This reduces the accuracy of the simulation, but also greatly reduces the memory consumption [Nic25]. We ran the simulation for a total of 100 million traces, the results are in Figure 3. Although the 10^{-5} threshold for the p -value is exceeded at certain intervals, it stays below the threshold for the majority of the time. The PROLEAD

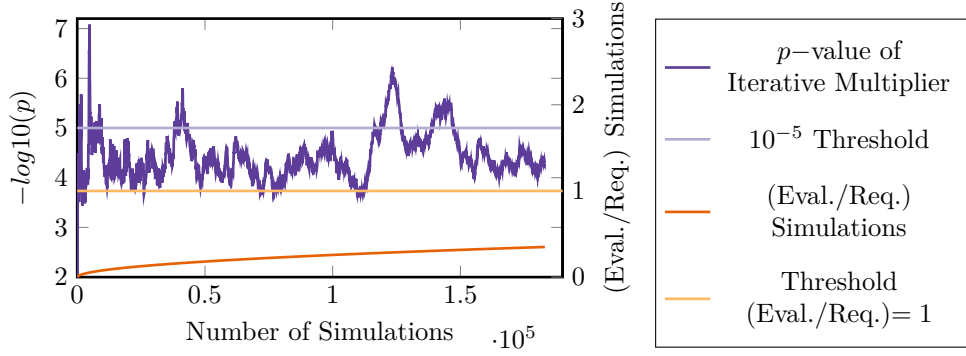


Figure 2: Incomplete first-order Prolead simulation of the first-order 15-bit masked multiplier, with the adder included. Due to running out of memory, only 180 000 simulations are performed. The p -value is below the threshold of 10^{-5} in the end, though the threshold of (Eval./Req.)= 1 for statistical confidence is not reached.

documentation states that when there are a large number of probing sets (the number of positions where probes can be placed, which increases as the circuit grows more complex), the chance of false positives increases [Nic25]. The documentation also lists 100 million as the minimum number of simulations when running in compact mode. As the p -value does not continuously increase as in Figure 1, we concluded that the temporary exceeding of the threshold is a false-positive.

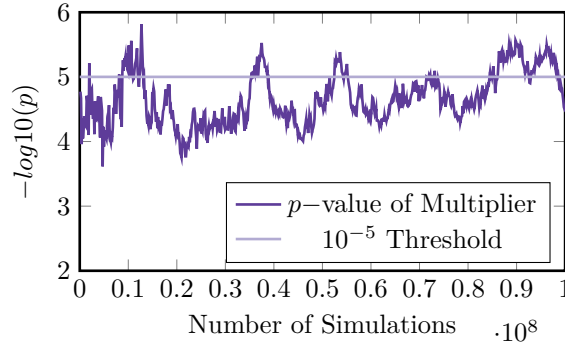


Figure 3: First-order Prolead simulation of the first-order 15-bit masked multiplier, with the adder included, in compact mode. In compact mode, Prolead uses the Hamming weight of signals to check for leakages, rather than the actual signal value. Prolead does not output the (Eval./Req.) for statistical confidence when running in compact mode. Instead we run more simulations, for a total of 100 million.

For further analysis, we split the circuit, analyzing the Sklansky adder separately from the masked iterative multiplier. The PROLEAD evaluation of the Sklansky adder is shown in Figure 4, where we performed a total of 5 million simulations. The p -value stays below the threshold except for a brief moment after around 3.5 million traces, and does not rise continuously. For this reason, we conclude that the adder does not leak. The results for the masked iterative multiplier with the adder excluded are shown in Figure 5, also with a total of 5 million simulations. At no point does the p -value exceed the threshold. We thus conclude that our masked design does not exhibit leakages.

An aspect that we exclude from our side-channel analysis is the RNG used to generate the randomness for the sampling, as well as the RNG to create the randomness to refresh

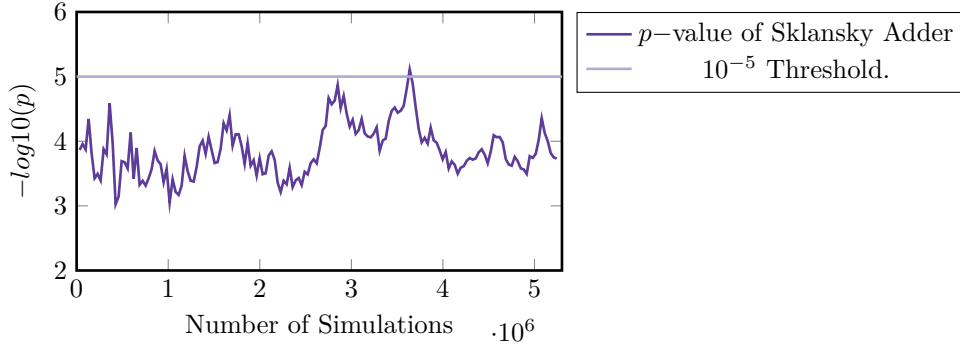


Figure 4: First-order Prolead simulation of the first-order 16-bit masked Sklansky adder with 5 million traces. The (Eval./Req.) simulations plot for statistical significance is not depicted, as the requirement is reached after just 30 000 simulations.

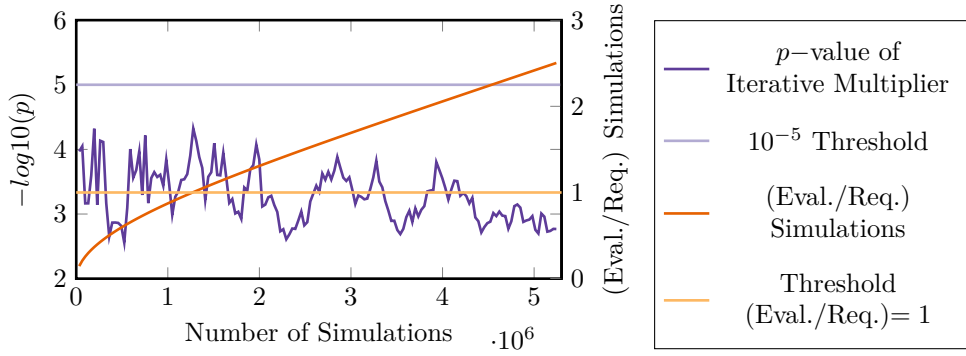


Figure 5: First-order Prolead Simulation of the first-order 15-bit masked multiplier, with the 16-bit masked Sklansky excluded. The required number of simulations for statistical confidence is reached after around 1 200 000 simulations.

the gadgets. Especially the RNG for the sampled values (Line 9 in Algorithm 6) would also require side-channel protection. However, these parts aren’t unique to our design, and are a requirement for all masked sampling implementations. For this reason, we consider the RNG to be out of scope of our analysis.

6 Conclusion

In this work, we presented multiple hardware implementations for fixed-weight sampling, a key part of PQC algorithms such as NTRU-HPS and Classic McEliece. We optimized each implementation with different targets in mind: Maximum performance, minimal area, or side-channel security. Our designs outperform existing hardware implementations of fixed-weight sampling for NTRU-HPS and Streamlined NTRU Prime in the literature, having either better performance, lower area, or both. Our low-area implementations are also the only fixed-weight sampling methods that do not require any memory. To our knowledge, our masked design is the first side-channel protected hardware implementation for fixed-weight sampling for NTRU-like schemes in the literature. For Classic McEliece, our designs outperform previous implementations that utilize sorting, but cannot keep-up with rejection-based approaches that are designed for sparse fixed-weight sampling.

Our work shows that the performance and efficiency improvements of the new fixed-weight sampling algorithm from [FSL24] also extend to hardware implementations. We

thus support the conclusion from [FSL24] that this new approach should be considered for future standardization of PQC algorithms that utilize fixed-weight sampling, especially when the to-be-sampled vectors are dense. Our work also confirms statements from [FAB⁺25] that this type of fixed-weight sampling algorithm is less effective when applied to spares vectors such as those found code-based PQC algorithms.

With regards to future work, one aspect of interest which we did not address are ASIC implementations. An ASIC synthesis of our low-area design would be particularly interesting, as the design requires no memory or multiplier blocks, both of which can consume significant silicon area. As our low-area design is the only implementation that does not require any memory, this could result in a very small ASIC footprint compared to other designs. Our masked fixed-weight sampling design can also be used to enable side-channel secure hardware implementations of NTRU-HPS, Streamlined NTRU Prime and Classic McEliece encapsulation and key generation, as these operations require fixed-weight sampling. A further interesting future work would be the application of other masking schemes, such as domain-oriented masking [GMK16], to our hardware designs.

7 Acknowledgements

This work was supported by the Federal Ministry of Research, Technology and Space (BMFTR) of the Federal Republic of Germany (grant 16KIS1572K, SASVI), as well as the European Commission under the grant agreement number 101070374 (CONVOLVE). This work was supported by the European Cybersecurity Competence Centre under the European Union’s Horizon Europe research and innovation program under grant agreement number 101225859 (FOCAL).

References

- [ABB⁺22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillipe Gaborit, Shay Gueron, Tim Guneyasu, Carlos Aguilar-Melchor, Rafael Misoczki, Edoardo Persichetti, Nicolas Sendrier, Jean-Pierre Tillich, Gilles Zémor, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann. BIKE. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [ABC⁺22] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- [ABPS23] Francesco Antognazza, Alessandro Barengi, Gerardo Pelosi, and Ruggero Susella. A Flexible ASIC-Oriented Design for a Full NTRU Accelerator. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference, ASPDAC '23*, page 591–597, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3566097.3567916.
- [BBC⁺20] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, Chitchanok Chuengsatiansup, Tanja Lange, Adrian Marotzke, Bo-Yuan Peng, Nicola

- Tuveri, Christine van Vredendaal, and Bo-Yin Yang. NTRU Prime. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [BCLvV17] Daniel J. Bernstein, Chitchanok Chuengsatiansup, Tanja Lange, and Christine van Vredendaal. NTRU prime: Reducing attack surface at low cost. In Carlisle Adams and Jan Camenisch, editors, *SAC 2017*, volume 10719 of *LNCS*, pages 235–260. Springer, Cham, August 2017. doi:10.1007/978-3-319-72565-9_12.
- [BG22] Florian Bache and Tim Güneysu. Boolean masking for arithmetic additions at arbitrary order in hardware. *Applied Sciences*, 12(5), 2022. URL: <https://www.mdpi.com/2076-3417/12/5/2274>, doi:10.3390/app12052274.
- [BGI⁺18] Roderick Bloem, Hannes Groß, Rinat Iusupov, Bettina Könighofer, Stefan Mangard, and Johannes Winter. Formal verification of masked hardware implementations in the presence of glitches. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 321–353. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78375-8_11.
- [BLH⁺25] Fabian Buschkowski, Georg Land, Niklas Höher, Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. HADES: Automated Hardware Design Exploration for Cryptographic Primitives. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):1–45, Sep. 2025. URL: <https://tches.iacr.org/index.php/TCHES/article/view/12402>, doi:10.46586/tches.v2025.i4.1-45.
- [bri25] brian m. carlson, Taylor Blau. Post-quantum security for SSH access on GitHub. <https://github.blog/engineering/platform-security/post-quantum-security-for-ssh-access-on-github/>, 2025. [Online; accessed 11-11-2025].
- [CCD⁺22] Po-Jen Chen, Tung Chou, Sanjay Deshpande, Norman Lahr, Ruben Niederhagen, Jakub Szefer, and Wen Wang. Complete and improved FPGA implementation of classic McEliece. *IACR TCHES*, 2022(3):71–113, 2022. doi:10.46586/tches.v2022.i3.71-113.
- [CDH⁺20] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M. Schanck, Peter Schwabe, William Whyte, Zhenfei Zhang, Tsunekazu Saito, Takashi Yamakawa, and Keita Xagawa. NTRU. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware Private Circuits: From Trivial Composition to Full Verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021. doi:10.1109/TC.2020.3022979.
- [CMM⁺24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness generation for secure hardware masking - unrolled trivium to the rescue. *CiC*, 1(2):4, 2024. doi:10.62056/akdkp2fgx.

- [DMG21] V Dang, Kamyar Mohajerani, and Kris Gaj. High-speed hardware architectures and fair FPGA benchmarking of CRYSTALS-Kyber, NTRU, and Saber. In *The NIST Third Standardization Conference*, pages 1–48, 2021.
- [FAB⁺25] Décio Luiz Gazzoni Filho, Gora Adj, Slim Bettaieb, Alessandro Budroni, Jorge Chávez-Saab, and Francisco Rodríguez-Henríquez. Sota voce: Low-noise sampling of sparse fixed-weight vectors. Cryptology ePrint Archive, Paper 2025/1299, 2025. URL: <https://eprint.iacr.org/2025/1299>.
- [FGM⁺18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR TCHES*, 2018(3):89–120, 2018. URL: <https://tches.iacr.org/index.php/TCHES/article/view/7270>, doi:10.13154/tches.v2018.i3.89-120.
- [FMJ25] Markus Friedl, Jan Mojzis, and Simon Josefsson. Secure Shell (SSH) Key Exchange Method Using Hybrid Streamlined NTRU Prime sntrup761 and X25519 with SHA-512: sntrup761x25519-sha512. Internet-Draft draft-ietf-sshm-ntruprime-ssh-06, Internet Engineering Task Force, September 2025. Work in Progress. URL: <https://datatracker.ietf.org/doc/draft-ietf-sshm-ntruprime-ssh/06/>.
- [FSL24] Décio Luiz Gazzoni Filho, Tomás S. R. Silva, and Julio Cesar López-Hernández. Efficient isochronous fixed-weight sampling with applications to NTRU. *CiC*, 1(2):14, 2024. doi:10.62056/a6n59qgxq.
- [GMK16] Hannes Gross, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. Cryptology ePrint Archive, Report 2016/486, 2016. URL: <https://eprint.iacr.org/2016/486>.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003. doi:10.1007/978-3-540-45146-4_27.
- [KAA21] Emre Karabulut, Erdem Alkim, and Aydin Aysu. Single-Trace Side-Channel Attacks on w -Small Polynomial Sampling: With Applications to NTRU, NTRU Prime, and CRYSTALS-DILITHIUM. In *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 35–45, 2021. doi:10.1109/HOST49136.2021.9702284.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999. doi:10.1007/3-540-48405-1_25.
- [KLRGB23] Markus Krausz, Georg Land, Jan Richter-Brockmann, and Tim Güneysu. A holistic approach towards side-channel secure fixed-weight polynomial sampling. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 94–124. Springer, Cham, May 2023. doi:10.1007/978-3-031-31371-4_4.
- [LMRGB24] Georg Land, Adrian Marotzke, Jan Richter-Brockmann, and Tim Güneysu. Gadget-based masking of streamlined NTRU prime decapsulation in hardware. *IACR TCHES*, 2024(1):1–26, 2024. doi:10.46586/tches.v2024.i1.1-26.

- [Mar20] Adrian Marotzke. A Constant-Time Full Hardware Implementation of Streamlined NTRU Prime. In Pierre-Yvan Liardet and Nele Mentens, editors, *Smart Card Research and Advanced Applications - 19th International Conference, CARDIS 2020, Virtual Event, November 18-19, 2020, Revised Selected Papers*, volume 12609 of *Lecture Notes in Computer Science*, pages 3–17. Springer, 2020. doi:10.1007/978-3-030-68487-7_1.
- [Mar25] Adrian Marotzke. *Secure and efficient hardware implementations of NTRU Prime*. PhD thesis, Hamburg University of Technology, 2025. <https://doi.org/10.15480/882.15172>.
- [McD14] John H McDonald. *Handbook of biological statistics*. Sparky House Publishing, Baltimore, Maryland. USA, 2014.
- [MM22] Nicolai Müller and Amir Moradi. PROLEAD A probing-based hardware leakage detection tool. *IACR TCHES*, 2022(4):311–348, 2022. doi:10.46586/tches.v2022.i4.311-348.
- [Mul23] Mullvad. Stable quantum-resistant tunnels in the app, 2023. [Online; accessed 11-11-2025]. URL: <https://mullvad.net/en/blog/2023/4/6/stable-quantum-resistant-tunnels-in-the-app>.
- [Nic25] Nicolai Müller. Prolead Wiki. <https://github.com/ChairImpSec/PROLEAD/wiki/Results#interpretation>, 2025. [Online; accessed 11-11-2025].
- [NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihang Qing, and Ninghui Li, editors, *ICICS 06*, volume 4307 of *LNCS*, pages 529–545. Springer, Berlin, Heidelberg, December 2006. doi:10.1007/11935308_38.
- [Ope22] OpenSSH Team. OpenSSH Release 9.0. <https://www.openssh.org/txt/release-9.0>, 2022. [Online; accessed 11-11-2025].
- [PMT⁺23] Bo-Yuan Peng, Adrian Marotzke, Ming-Han Tsai, Bo-Yin Yang, and Ho-Lin Chen. Streamlined NTRU prime on FPGA. *Journal of Cryptographic Engineering*, 13(2):167–186, June 2023. doi:10.1007/s13389-022-00303-z.
- [RBFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. VERICA - verification of combined attacks automated formal verification of security against simultaneous information leakage and tampering. *IACR TCHES*, 2022(4):255–284, 2022. doi:10.46586/tches.v2022.i4.255-284.
- [RM25] Hemin Rahimi and Amir Moradi. One More Motivation to Use Evaluation Tools This Time for Hardware Multiplicative Masking of AES. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. doi:10.23919/DATE64628.2025.10993221.
- [Skl60] J. Sklansky. Conditional-sum addition logic. *IRE Transactions on Electronic Computers*, EC-9(2):226–231, 1960. doi:10.1109/TEC.1960.5219822.
- [SM15] Tobias Schneider and Amir Moradi. Leakage assessment methodology - A clear roadmap for side-channel evaluations. In Tim Güneysu and Helena Handschuh, editors, *CHES 2015*, volume 9293 of *LNCS*, pages 495–513. Springer, Berlin, Heidelberg, September 2015. doi:10.1007/978-3-662-48324-4_25.

-
- [WSN18] Wen Wang, Jakub Szefer, and Ruben Niederhagen. FPGA-Based Niederreiter Cryptosystem Using Binary Goppa Codes. In Tanja Lange and Rainer Steinwandt, editors, *Post-Quantum Cryptography*, pages 77–98, Cham, 2018. Springer International Publishing.
- [Xil18] Xilinx, Inc. *UG479: 7 Series DSP48E1 Slice*, 1.10 edition, March 2018. https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1.
- [ZZC⁺23] Yihong Zhu, Wenping Zhu, Chen Chen, Min Zhu, Zhengdong Li, Shaojun Wei, and Leibo Liu. Mckeycutter: A High-throughput Key Generator of Classic McEliece on Hardware. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2023. doi:10.1109/DAC56929.2023.10247918.

A Sage Script for calculating FIFO depth

```

1 L = 16
2 #L = 19
3
4 # NTRU for degree n, samples a vector of length n-1.
5 # NTRU Prime and Classic McEliece, for degree n,
6 # sample a vector of length n.
7 # As the code below assumes vector length of n-1, the
8 # NTRU Prime and Classic McEliece degrees
9 # are increased by 1 in the n_array
10 # to compensate for this.
11 n_array = [509,677,762,821,1278, 8193]
12
13 # Different FIFO depths.
14 fifo_depth_array = [16,32,64,128,256]
15
16 for n in n_array:
17     print("Array length: ", n-1)
18
19     min_prop=min([(1-(2^L%(n-1-i)))/2^L for i in range(n-1)])
20     print("Worst case rejection probability: ", RR(1-min_prop))
21
22     avg_prop=mean([(1-(2^L%(n-1-i)))/2^L for i in range(n-1)])
23     print("Mean rejection probability: ", RR(1-avg_prop))
24
25     for fifo_depth in fifo_depth_array:
26         f = sum(binomial(n-1,k)*(1-min_prop)**k*min_prop**(n-1-k)
27                 for k in range(fifo_depth+1,n-1))
28         print("Upper bound of probability of exceeding FIFO
29               depth of ", fifo_depth, " is ", RR(f), ", 2^", RR(
30                 log(f,2)), sep='')
31
32         f_avg = sum(binomial(n-1,k)*(1-avg_prop)**k*avg_prop**(n-1-k)
33                     for k in range(fifo_depth+1,n-1))
34         print("Estimated probability (based on mean rejection
35               probability) of exceeding FIFO depth of ",
36               fifo_depth, " is ", RR(f_avg), ", 2^", RR(log(f_avg,
37               2))), sep='')
38     print(" ")

```

Listing 1: A Sage script to compute the probability for a given sampled vector length that the number of rejected samples during the first pass exceeds a certain FIFO depth. We ran this with SageMath 9.2. In addition to using the worst-case probability for a rejection, the script also computes an estimated probability using the average rejection probability.